

Hub Cultura Builder — camada B2B

O que precisa ser construído para a empresa existir dentro do produto: onboarding self-service do time, identidade completa do usuário, progressão por construção (o “Strava do build”), painel da empresa, clusterização no admin e feed com escopo. Modelo de dados, telas, endpoints e critérios de aceite por módulo.

NESTE DOCUMENTO

- | | |
|--|------------------------------------|
| 0. Como usar / princípios inegociáveis | 6. M4 — Painel da empresa |
| 1. O que estamos construindo | 7. M5 — Admin CB e audiências |
| 2. Modelo de dados (base) | 8. M6 — Feed com escopo |
| 3. M1 — Onboarding de empresa | 9. Permissões e privacidade |
| 4. M2 — Identidade do usuário | 10. Backlog recomendado |
| 5. M3 — Progressão (Strava do build) | 11. Ordem de implementação |
| | 12. Perguntas para a primeira call |

0. Como usar este documento

Este documento descreve **o que** precisa existir e **por quê**, com modelo de dados, telas, endpoints e critérios de aceite. Ele **não** conhece o schema real do hub — foi escrito a partir da API pública (/api/subscription/plans , /api/aulas , /api/challenger/projects) e do que já está em produção. Sempre que houver conflito entre o que está aqui e o que existe no código, **o código ganha**: a regra é adaptar nomes/tipos ao que já existe, não criar estrutura paralela.

Sequência sugerida para o agente de código:

1. Ler o repositório e produzir um **mapa do que já existe** para cada conceito deste documento (usuário, progresso de aula, projeto/challenger, badges, onboarding, admin, notificações, billing).
2. Devolver um **plano de migração** (tabelas novas x colunas em tabelas existentes) antes de escrever código.

3. Implementar **um módulo por PR**, na ordem da seção 11, cada um atrás de feature flag.
4. Cada PR entrega: migration + endpoints + tela + seed de teste + os critérios de aceite marcados.

Princípios inegociáveis

#	PRINCÍPIO	CONSEQUÊNCIA PRÁTICA
1	Nenhum segundo login	A camada de empresa consome a identidade que já existe (Stack Auth). Nada de app separado com auth próprio — isso cria para sempre o problema do usuário com duas contas.
2	Estender, nunca duplicar	Builders Projects vira registro de builds. Badges viram faixas. O motor de onboarding vira o perfil. O admin atual recebe as novas abas.
3	B2C continua funcionando	Todo campo de empresa é <i>nullable</i> . Usuário sem empresa não pode perceber diferença nenhuma.
4	Todo evento relevante vira registro	Progressão, painel da empresa, admin e score leem do mesmo ledger de eventos . Nada de contador denormalizado como fonte da verdade.
5	Adoção, não recriação de conta	Funcionário que já tem conta pessoal no hub é vinculado à empresa sem perder progresso, faixa ou histórico .
6	Dado individual é sensível	Assiduidade por pessoa: só admin da própria empresa e CB. Sponsor vê agregado por área. Ver seção 9.
7	Número não medido não aparece	Se o hub não mede, o painel não exhibe. Sem placeholder, sem estimativa inventada.

1. O que estamos construindo

O hub hoje é **single-player B2C**: pessoa entra, assina, assiste aula, publica projeto. A venda B2B já acontece fora do produto — contrato fechado, e a equipe **cria e libera cada membro na mão**. Isso não escala e, pior, o contrato não existe dentro do produto: não há empresa, não há assento, não há visão do patrocinador.

A camada B2B adiciona seis coisas:

1. **Onboarding de empresa self-service** — a empresa cadastra o próprio time (a prioridade número 1 deste documento).
2. **Identidade completa do usuário** — quem é, de que empresa, que área, que cargo, que senioridade, que nível.
3. **Progressão gamificada por construção** — o “Strava do build”: a pessoa sobe de nível ao *entregar*, não ao assistir.
4. **Painel da empresa** — o dono do contrato vê quem está evoluindo e quem não está.
5. **Admin CB com clusterização** — filtrar a base inteira por nível/área/cargo/empresa para produzir conteúdo e turmas por segmento.
6. **Feed com escopo** — post visível para todo mundo ou só para a empresa.

2. Modelo de dados (base)

Postgres. Nomes sugeridos; adaptar à convenção do repositório. `user_id` referencia o identificador de usuário já usado pelo hub (aparentemente UUID de texto, conforme `ownerId` em `/api/challenger/projects`).

```

-- 2.1 Empresa e estrutura
create table companies (
  id          bigserial primary key,
  name        text not null,
  legal_name  text,
  cnpj        text unique,
  logo_url    text,
  industry    text,
  size_band   text,                -- '1-10','11-50','51-200','201-1000','1000+'
  seats_total int not null default 0,
  contract_start date,
  contract_end date,
  status      text not null default 'active', -- active | paused | churned | trial
  sponsor_user_id text,                -- dono do contrato (visão read-only)
  success_owner_id text,              -- responsável CB pela conta
  settings    jsonb not null default '{}'::jsonb, -- flags: feed_enabled, ranking_enabled
  created_at  timestampz not null default now()
);

create table company_email_domains (      -- domínios que a empresa reivindica
  id          bigserial primary key,
  company_id  bigint not null references companies(id) on delete cascade,
  domain      text not null,            -- 'empresa.com.br'
  auto_join   boolean not null default false, -- entrada automática por domínio
  verified_at timestampz,
  unique (domain)
);

create table departments (
  id          bigserial primary key,
  company_id  bigint not null references companies(id) on delete cascade,
  name        text not null,
  lead_user_id text,                    -- líder da área (mini-painel)
  unique (company_id, name)
);

-- seed padrão por empresa (editável no wizard):
-- Comercial, Marketing, Financeiro, RH, Jurídico, Operações, TI/Produto, Atendimento, Diretoria

-- 2.2 Vínculo pessoa <=> empresa (tabela, não coluna: a pessoa pode trocar de empresa)
create table company_members (
  id          bigserial primary key,
  company_id  bigint not null references companies(id) on delete cascade,
  user_id     text not null,
  department_id bigint references departments(id) on delete set null,
  org_role    text not null default 'member', -- member | org_admin | sponsor | lead
  job_title   text,                          -- cargo livre: "Analista de Marketing Pleno"
  job_family  text,                          -- enum normalizado, ver 4.1
  seniority   text,                          -- enum, ver 4.1
  manager_user_id text,

```

```

status          text not null default 'active', -- invited | active | suspended | removed
seat_consumed   boolean not null default true,
joined_at       timestampz,
removed_at      timestampz,
unique (company_id, user_id)
);
create index on company_members (user_id) where status = 'active';

-- 2.3 Convites
create table company_invites (
  id              bigserial primary key,
  company_id      bigint not null references companies(id) on delete cascade,
  email           citext not null,
  name            text,
  department_id   bigint references departments(id),
  job_title       text,
  job_family      text,
  seniority       text,
  manager_email   citext,
  org_role        text not null default 'member',
  token           text not null unique,
  status          text not null default 'pending', -- pending|accepted|expired|revoked|bounced
  batch_id        uuid,                          -- agrupa um CSV/lote
  invited_by      text,
  reminders_sent  int not null default 0,
  expires_at      timestampz not null,
  accepted_at     timestampz,
  accepted_user_id text,
  created_at      timestampz not null default now()
);
-- um convite vivo por e-mail por empresa (índice parcial; `citext` exige a extensão citext,
-- senão usar text + lower(email) no índice)
create unique index company_invites_live_uniq
  on company_invites (company_id, email)
  where status in ('pending','accepted');

-- 2.4 Entitlement por empresa (acesso ao conteúdo pelo contrato, não pela assinatura pessoal)
create table company_entitlements (
  id              bigserial primary key,
  company_id      bigint not null references companies(id) on delete cascade,
  feature_slug    text not null, -- reaproveitar os slugs de `features` já existentes nos plan
  starts_at      timestampz not null default now(),
  ends_at        timestampz,
  unique (company_id, feature_slug)
);

```

Resolução de acesso (regra única, um só lugar no código):

```
acesso(user, feature) =  
    entitlement_individual(user, feature)                -- plano pessoal atual  
    OR entitlement_da_empresa(company_ativa(user), feature) -- contrato B2B vigente
```

Nunca duplicar essa regra em rota/middleware/componente: um helper `hasFeature(userId, slug)` e todo mundo chama ele.

3. Módulo 1 — Onboarding de empresa (prioridade 1)

Problema hoje: vendemos, e a equipe cria/libera cada pessoa manualmente, uma por uma. **Objetivo:** a empresa cadastra o próprio time em uma sessão de 10 minutos, e a CB só acompanha.

3.1 Fluxo completo

Passo 0 — CB cria a conta da empresa (admin do hub, 2 minutos) Nome, CNPJ, plano, nº de assentos, vigência, domínios de e-mail, quem é o sponsor, quem é o org_admin, features do contrato. Gera um **link de setup** (magic link, validade 14 dias) enviado ao org_admin.

Passo 1 — Wizard de setup da empresa (org_admin, self-service)

ETAPA	O QUE ACONTECE
1. Empresa	Confirma nome, logo, setor, porte. Escolhe se o feed interno fica ligado, se o ranking interno fica ligado.
2. Áreas	Lista de áreas vem pré-semeada; org_admin edita, remove, adiciona e marca o líder de cada área.
3. Pessoas	Três formas de entrada, ver 3.2. Validação linha a linha com pré-visualização.
4. Revisão e disparo	Mostra: X convites, Y assentos livres, quem já tem conta no hub. Dispara.

Passo 2 — A pessoa recebe o convite (e-mail; opcionalmente WhatsApp) Clica → tela de aceite com nome da empresa, o que ela ganha e o aviso de privacidade (seção 9) → login/cadastro Stack Auth →

onboarding pessoal curto (3 perguntas) com área e cargo já preenchidos pelo convite → cai direto na trilha da faixa 1.

Passo 3 — Acompanhamento (org_admin e CB veem o mesmo painel de ativação) Convites enviados / aceitos / pendentes / expirados, com botão de reenviar. Lembretes automáticos em D+2, D+7, D+14. Assentos ociosos aparecem em destaque.

3.2 Três formas de cadastrar pessoas

1. **Colar e-mails** — textarea, um por linha ou separado por vírgula. Rápido, sem metadados.
2. **CSV** — o caminho principal. Template para download:
`nome,email,area,cargo,senioridade,email_do_lider,papel`
 - Validação por linha antes de importar: e-mail inválido, duplicado no arquivo, já convidado, já membro de outra empresa, área inexistente (oferecer criar), assentos insuficientes.
 - Tela de pré-visualização mostrando erros por linha e permitindo corrigir na própria tabela.
 - Importação é **idempotente por batch_id** : reenviar o mesmo arquivo não duplica convite nem consome assento duas vezes.
3. **Link de entrada por domínio** — link único da empresa que só aceita e-mails @domínio-verificado , com teto de assentos e opção de exigir aprovação do org_admin. Resolve empresas grandes que não querem montar lista.

Opcional (`settings.provision_mode = 'accounts'`): em vez de convite, **criar as contas direto** e enviar magic link de primeiro acesso. O aceite de termos/privacidade continua obrigatório no primeiro login.

3.3 Regras de assento

- Assento é consumido **no aceite**, não no envio do convite.
- Convite pendente faz **reserva leve** de assento (configurável, padrão 7 dias). Ao expirar, devolve.
- Assentos esgotados: bloqueia novo convite com mensagem clara ("15/15 usados — libere um assento ou fale com a CB") e botão "solicitar mais assentos" que abre chamado para o Success.
- Remover pessoa: devolve o assento, mantém histórico/faixa/projetos, e a conta dela **rebaixa para o plano individual que tiver (ou free)** — sem apagar nada.
- Trocar pessoa de área: não mexe em assento nem em progresso.

3.4 Adoção de conta existente (caso real e frequente)

Funcionário já é aluno B2C. O convite **vincula** a conta existente: - não cria conta nova, não pede nova senha; - mantém todo o progresso, faixas, projetos, streak; - se ele tem assinatura paga individual ativa, mostrar aviso ("sua empresa agora cobre seu acesso") e **não** cancelar automaticamente — sinalizar para o Success tratar; - se ele já pertence a outra empresa ativa, o convite exige confirmação explícita e registra a troca (`company_members.status = 'removed'` na anterior).

3.5 Endpoints

POST	/api/admin/companies	cria empresa (CB)
POST	/api/admin/companies/:id/setup-link	gera magic link do wizard
GET	/api/org/:companyId/overview	assentos, ativação, status
POST	/api/org/:companyId/departments	CRUD de áreas
POST	/api/org/:companyId/invites	lote de convites (array)
POST	/api/org/:companyId/invites/preview	valida CSV e devolve erros por linha
POST	/api/org/:companyId/invites/:id/resent	
DELETE	/api/org/:companyId/invites/:id	revoga
POST	/api/org/:companyId/join-link	cria/rotaciona link por domínio
GET	/api/invites/:token	dados públicos do convite (empresa, expiração)
POST	/api/invites/:token/accept	consome assento, cria vínculo
DELETE	/api/org/:companyId/members/:userId	remove e devolve assento

3.6 Critérios de aceite

- ☐ Org_admin cadastra 15 pessoas por CSV, com áreas e cargos, sem nenhuma intervenção da CB.
 - ☐ Arquivo com erro mostra os erros por linha e permite corrigir antes de importar; nada é importado pela metade.
 - ☐ Reimportar o mesmo CSV não gera convite duplicado nem consome assento a mais.
 - ☐ Aluno B2C existente aceita o convite e mantém progresso, faixa e projetos.
 - ☐ Assento é liberado ao remover a pessoa e ao expirar o convite.
 - ☐ Membro ativo acessa o conteúdo Premium pelo contrato da empresa, sem assinatura individual.
 - ☐ Empresa pausada/vencida (`status != 'active'` ou `contract_end < hoje`) rebaixa todos os assentos automaticamente, sem apagar dado.
 - ☐ Painel de ativação mostra, em tempo real, convidados / aceitos / ativos na semana.
-

4. Módulo 2 — Identidade do usuário

Objetivo: responder, para qualquer pessoa da base, **quem é, de onde vem, o que faz, em que nível está** — e conseguir agrupar por isso (seção 7).

4.1 Campos

Tabela `user_profiles` (1:1 com usuário), além do que já existe:

CAMPO	TIPO	ORIGEM	OBSERVAÇÃO
job_family	enum	convite ou onboarding	Marketing, Vendas, Financeiro, RH, Jurídico, Operações, TI/Produto, Atendimento, Diretoria, Educação, Outro
seniority	enum	convite ou onboarding	estagiário, analista, especialista, coordenador, gerente, diretor, C-level, sócio/dono
job_title	texto	livre	cargo como a pessoa escreve
tech_level	enum	auto-declarado	não-técnico, uso ferramentas no-code, programa pouco, programa
ai_level_declared	enum	onboarding	nunca usei, uso ChatGPT, automatizo tarefas, construo apps
goal	enum	onboarding	aprender pra mim, ensinar meu time, minha empresa pediu, construir um produto, mudar de carreira
tools_used	texto[]	onboarding	ChatGPT, Claude, Cursor, n8n, Make, Zapier, Excel/Sheets, ...
github_username	texto	OAuth	ver 5.4
linkedin_url	texto	perfil	usado no selo de faixa
city / state	texto	perfil	recorte regional para lives
timezone	texto	auto	agendamento de turmas

CAMPO	TIPO	ORIGEM	OBSERVAÇÃO
visibility	enum	usuário	public , company_only , private — controla o que aparece no feed/perfil público

Regra de origem: gravar `source` (`self` , `company` , `inferred`) por campo, ou uma tabela `user_profile_fields(user_id, field, value, source, updated_at)` . O dado declarado pela empresa **não** é sobrescrito por auto-declaração posterior sem confirmação, e vice-versa. Isso evita o clássico “o CSV do RH dizia gerente, a pessoa mudou pra analista e o painel do sponsor ficou errado”.

4.2 Onboarding pessoal

Reaproveitar o motor de onboarding existente (o mesmo que já produziu as 164 respostas em `admin` → `onboarding-insights`), com duas mudanças:

1. **Perguntar só o que falta.** Convidado por empresa já chega com empresa, área e cargo — não perguntar de novo. Máximo 3 perguntas: nível de IA, objetivo, o que quer construir nos próximos 30 dias.
2. **Pergunta de roteamento** (para todo mundo, inclusive B2C): “você quer trazer sua empresa?” → se sim, captura empresa/cargo e cai numa fila de expansão do comercial. É o único caminho de entrada inbound de empresa que temos.

4.3 Perfil visível

- **Perfil público:** nome, avatar, faixa, builds publicados, badges, cidade, streak. Empresa só aparece se `visibility = public` e a empresa permitir (`settings.show_company_on_profile`).
- **Perfil interno (colegas da mesma empresa):** acrescenta área, cargo, builds internos.
- **Perfil administrativo (org_admin da empresa e CB):** tudo, incluindo atividade.

5. Módulo 3 — Progressão: o “Strava do build”

A referência é explícita: **Strava para quem constrói com IA**. Não é curso com barra de progresso — é registro público do que a pessoa entregou, com constância visível.

5.1 Duas camadas que não se misturam

CAMADA	O QUE É	COMO MUDA	PARA QUE SERVE
Faixa (nível)	Operador → Construtor → Builder → Embaixador	Só por artefato entregue e revisado por humano . Nunca por XP, nunca por aula assistida.	Selo de capacidade. É o que vale para a empresa e para o LinkedIn.
Atividade (pontos, streak, liga)	Motor de hábito semanal	Automática, por evento	Ranking, constância, sinal de risco, feed

Se pontos derem faixa, a faixa vira participação. Manter separado é o que faz o sistema significar alguma coisa.

Critérios das faixas (rubrica visível na própria tela de submissão):

- 1. **Operador** — automatizou uma tarefa do próprio trabalho. Evidência: o que era, quanto tempo levava, link/print do que roda hoje.
- 2. **Construtor** — construiu algo que **outra pessoa** usa. Evidência: quem usa, desde quando, link.
- 3. **Builder** — algo em produção na área, com impacto medido. Evidência: link, número antes/depois, quem aprovou.
- 4. **Embaixador** — ensinou e certificou outra pessoa internamente. Evidência: quem, o quê, artefato do certificado.

5.2 Ledger de eventos (base de tudo)

```

create table activity_events (
  id          bigserial primary key,
  user_id     text not null,
  company_id  bigint references companies(id),    -- snapshot da empresa no momento do evento
  type        text not null,                     -- ver tabela abaixo
  ref_type    text,                             -- 'aula','curso','project','artifact','repo',
  ref_id      text,
  points      int not null default 0,
  occurred_at timestamptz not null default now(),
  season_id   bigint references seasons(id),
  metadata    jsonb not null default '{} '::jsonb,
  dedupe_key  text unique                       -- ex: 'aula_concluida:user:123:aula:45'
);
create index on activity_events (user_id, occurred_at desc);
create index on activity_events (company_id, occurred_at desc);

```

Tudo (perfil, painel da empresa, admin, Builder Score, feed) lê daqui. `dedupe_key` único torna reprocessamento seguro.

5.3 Eventos e pontos v1

Pontos ficam em **tabela de configuração versionada** (`point_rules`), nunca hardcoded — vamos calibrar.

EVENTO	PONTOS	TETO	OBSERVAÇÃO
etapa_concluida	2	20/dia	granularidade que a API de aulas já expõe (completed_etapas)
aula_concluida	10	—	
curso_concluido	60	—	
projeto_submetido	0	—	pontua só quando aprovado (evita spam)
projeto_aprovado	150	—	usa o fluxo de aprovação que já existe em Builders Projects
deploy_verificado	100	1 por projeto	ver 5.4
github_conectado	50	1	
repo_vinculado	30	5	
semana_com_commit	25	1/semana	constância, não volume
build_usado_por_colega	200	—	evidência de uso confirmada por um colega da empresa — o evento mais valioso do sistema
artefato_aprovado (faixa)	300	—	
demo_no_builders_club	120	—	marcado pela CB
mentoria_dada / resposta_util	20	5/semana	marcado por quem recebeu
streak_semanal	30	1/semana	semana com ao menos 1 evento de construção

Antifraude: teto por tipo, dedupe_key , pontos de projeto só pós-aprovação, e **ranking padrão por atividade dos últimos 90 dias** (o total histórico existe, mas não é o que aparece no topo — senão

veterano trava o ranking pra sempre).

5.4 Sinais de construção (o que diferencia de LMS)

1. **GitHub OAuth** (escopo mínimo, leitura de repos públicos + metadados). Ao conectar: badge, e sincronização semanal contando semanas com commit/PR nos repos que a pessoa vinculou. Não precisa ler código.
2. **Verificação de deploy** — a pessoa informa a URL do build; o hub confirma propriedade por um dos caminhos: arquivo `/.well-known/culturabuilder.txt` com o token, meta tag `<meta name="cb-verify" content="...">`, ou DNS TXT. Depois disso, um *health check* semanal marca “no ar” — a galeria de builds mostra o que **está de pé**, não o que já esteve.
3. **Evidência de uso** — no formulário de artefato, a pessoa indica colegas que usam; o hub envia confirmação de um clique para eles. Confirmado, dispara `build_usado_por_colega`.
4. **Horas economizadas declaradas** — campo numérico simples no artefato; agregado vira métrica do painel do sponsor. Sempre rotulado como **declarado**, nunca como medido.

5.5 Temporadas e ligas

```
create table seasons (id bigserial primary key, name text, theme text, starts_on date, ends_on d
```

Ranking de atividade zera por temporada (trimestre); faixas e badges são permanentes. Rankings disponíveis: minha área, minha empresa, global, temporada. Empresa pode desligar ranking interno (`settings.ranking_enabled`).

5.6 Tela de perfil (o “Strava”)

Acima da dobra: faixa atual + **checklist do que falta para a próxima** (rubrica, não barra); streak de semanas construindo; heatmap de 12 meses; builds em produção com link e status “no ar”; horas economizadas declaradas; badges; posição na temporada.

5.7 Resumo semanal

E-mail (e WhatsApp, se houver opt-in) toda segunda: o que você construiu, streak, posição, o que falta para a próxima faixa, uma sugestão de próximo passo. É o loop de retenção — o mesmo papel que o resumo semanal cumpre no Strava.

5.8 Critérios de aceite

- ☐ Nenhuma faixa é concedida sem artefato aprovado por revisor humano, com registro de auditoria (quem, quando, feedback).
 - ☐ `belt_awards` é imutável: revogação só por registro administrativo separado, nunca deleção.
 - ☐ Reprocessar eventos não duplica pontos.
 - ☐ Perfil mostra faixa, streak, builds no ar e o que falta para subir.
 - ☐ GitHub conectado gera evento semanal sem intervenção manual.
 - ☐ URL de deploy não verificada não conta ponto.
-
-

6. Módulo 4 — Painel da empresa

Duas visões, dois papéis (ver seção 9): **org_admin** (opera, vê indivíduo) e **sponsor** (decide renovação, vê agregado).

6.1 Telas

Visão geral — assentos usados/livres/ativados; % ativos na semana e no mês; distribuição por faixa; builds em produção; horas economizadas declaradas; Builder Score e a **curva desde o dia zero** (nunca só o valor atual — o produto de renovação é a curva).

Pessoas (só org_admin) — tabela com filtros (área, faixa, cargo, senioridade, status) e colunas: última atividade, % da trilha, aulas concluídas, builds, faixa, streak. Ações: reenviar convite, trocar área, marcar líder, promover a org_admin, remover.

Áreas — comparativo entre áreas: adoção, builds, faixas, dores do diagnóstico. É o insumo do “quem precisa de qual conteúdo”.

Em risco — três listas prontas: convite não aceito há 7 dias; sem login há 14 dias; sem nenhum build em 30 dias. Cada linha com botão de cutucar (notificação no hub + e-mail/WhatsApp).

Builds da empresa — galeria interna, com filtro por área, link, status “no ar”, quem usa, impacto declarado. Vira o material da reunião trimestral.

Relatório — export PDF/CSV do trimestre e envio automático mensal ao sponsor.

6.2 Builder Score (índice da conta)

```
create table company_score_snapshots (
  id bigserial primary key,
  company_id bigint not null references companies(id) on delete cascade,
  period_start date not null,                -- semana ISO
  active_seats_pct numeric,                  -- assentos com login+ação na semana / total
  builds_in_production int,
  depts_with_build_pct numeric,
  ambassadors_count int,
  hours_saved_declared numeric,
  score numeric,
  formula_version text not null,
  created_at timestamptz not null default now(),
  unique (company_id, period_start)
);
```

Fórmula v1 (versionada, ajustável): $\text{score} = 40\% \cdot \text{assentos_ativos} + 30\% \cdot \text{áreas_com_build} + 20\% \cdot \text{builds_em_produção(normalizado)} + 10\% \cdot \text{embaixadores}$. Job semanal para toda empresa ativa; falha do job alerta o time. O primeiro snapshot é o **Score Zero**, gravado no início do contrato — é a linha de base da renovação.

6.3 Critérios de aceite

- ☐ Sponsor vê apenas a própria empresa e apenas dados agregados por área.
- ☐ Org_admin vê indivíduo, mas o aviso de privacidade foi aceito pelo usuário no convite.
- ☐ Snapshot semanal roda sozinho e nunca sobrescreve histórico.
- ☐ Export PDF sai idêntico ao que está na tela (sem número que só existe no PDF).
- ☐ Empresa com contrato vencido some dos jobs, mas mantém histórico consultável pela CB.

7. Módulo 5 — Admin CB: clusterização e audiências

Esse módulo é o que transforma a base em produto: hoje temos milhares de usuários e nenhuma forma de perguntar *“quem são os iniciantes de marketing dentro das empresas clientes?”*.

7.1 Filtros combináveis (sobre toda a base)

empresa · setor · porte · área/ job_family · seniority · faixa atual · tech_level · ai_level_declared · objetivo declarado · atividade (ativo 7/30/90 dias, nunca ativou) · tipo de conta (B2C / B2B / ambos) · plano · cidade/UF · data de entrada · tem projeto publicado? · tem GitHub conectado? · temporada de entrada · trilha em curso.

7.2 Audiências salvas

```
create table audiences (  
  id bigserial primary key,  
  name text not null,  
  description text,  
  filter jsonb not null,          -- os filtros, não a lista: é dinâmico  
  is_dynamic boolean default true,  
  created_by text,  
  created_at timestamptz default now()  
);
```

Uma audiência salva é reutilizável em: convidar para live, criar turma da Fundação, liberar conteúdo específico (gate por audiência), campanha de e-mail/WhatsApp, export CSV, e comparação de coorte ("marketing iniciante evolui mais devagar que financeiro iniciante?").

Caso de uso alvo, que precisa funcionar no dia 1: **"pessoas de Marketing, senioridade analista/coordenador, faixa Operador ou sem faixa, de qualquer empresa cliente"** → 1 clique → convidar para uma live específica daquele recorte.

7.3 Outras telas do admin CB

- **Contas** — lista de empresas com adoção, Score, tendência, risco de churn, próximo marco, dono do Success.
- **Conteúdo x cluster** — o que cada cluster consome, abandona e conclui. É o que decide o que gravar em vez de achismo.
- **Funil de ativação** — convidados → aceitos → primeira aula → primeiro build → primeira faixa, com corte por empresa e por área. O gargalo aparece aqui.
- **Fila de revisão de artefatos** — pendências por idade, SLA de 5 dias úteis, alerta de estouro.

7.4 Critérios de aceite

- ☐ Qualquer combinação de filtros devolve contagem em menos de 2 segundos na base atual.
- ☐ Audiência salva é dinâmica (recalcula), com opção de congelar snapshot.

- ☐ Export respeita LGPD: registro de quem exportou, quando e com qual filtro.

8. Módulo 6 — Feed com escopo (comunidade + canal da empresa)

8.1 Modelo

```
create table posts (
  id bigserial primary key,
  author_user_id text not null,
  company_id bigint references companies(id),      -- obrigatório quando visibility != 'public'
  department_id bigint references departments(id), -- opcional, escopo mais estreito
  visibility text not null default 'public',      -- public | company | department
  kind text not null default 'text',              -- text | ship | question | announcement
  body text not null,
  media jsonb default '[]'::jsonb,
  ref_type text, ref_id text,                     -- ex: kind='ship' aponta pro projeto
  pinned_until timestamptz,
  created_at timestamptz default now(),
  deleted_at timestamptz
);
create table post_comments (...);
create table post_reactions (...);
```

8.2 Regras

- Ao publicar, o usuário escolhe o alcance: **Comunidade** (todo o hub) ou **Minha empresa** (e, se fizer sentido, **Minha área**). Padrão: Comunidade para B2C, e configurável por empresa para B2B (`settings.default_post_scope`).
- Filtragem no **servidor**, sempre: `post company` só é consultável por membro ativo daquela empresa. Nunca filtrar no cliente.
- Post automático de “ship”**: quando um build é aprovado ou um deploy é verificado, oferecer publicação com um clique (opt-in, nunca automático sem consentimento) — é isso que faz o feed parecer o Strava e não um mural morto.
- `announcement` só por `org_admin`, com fixação no topo e notificação.
- Moderação: CB tem acesso a posts de empresa para moderação, com **registro de acesso** e a regra escrita nos termos. A empresa pode desligar o canal inteiro (`settings.feed_enabled = false`).

- No feed público, a empresa da pessoa só aparece se ela e a empresa permitirem (seção 4.3).
- Digest: resumo do canal da empresa por e-mail, semanal.

8.3 Critérios de aceite

- ☐ Post de empresa nunca aparece em nenhuma consulta pública, incluindo busca, notificação e API.
- ☐ Ex-membro perde acesso ao canal na hora da remoção.
- ☐ Empresa com feed desligado não expõe nenhuma rota do canal.

9. Permissões e privacidade

9.1 Papéis

PAPEL	ESCOPO	VÊ INDIVÍDUO	GERENCIA ASSENTOS	VÊ CONTEÚDO
member	próprio perfil + canal da empresa	só o próprio	não	pelo contrato
lead (líder de área)	sua área	sua área	não	pelo contrato
org_admin	empresa inteira	sim	sim	pelo contrato
sponsor	empresa inteira	não — só agregado por área	não	leitura do painel
reviewer (CB/professor)	fila de artefatos	só o que revisa	não	—
cb_admin	tudo	sim	sim	tudo

Papéis são **cumulativos** (o sponsor costuma também ser org_admin — então guarde os dois, não force escolha).

9.2 LGPD e confiança

- No aceite do convite, a pessoa lê e aceita, em texto claro: **o que a empresa vê** (progresso, faixa, builds, última atividade) e **o que a empresa não vê** (conteúdo de mensagens privadas, dados de assinatura pessoal anterior, respostas de onboarding marcadas como pessoais).
 - Base legal: execução de contrato entre CB e empresa; o titular pode sair da empresa no hub a qualquer momento (e volta ao plano individual).
 - Sponsor **não** vê assiduidade individual. Essa restrição é produto, não detalhe: é o que evita o hub virar ponto eletrônico e matar o engajamento.
 - Toda exportação e todo acesso administrativo a dado de empresa é registrado (`audit_log` : ator, ação, alvo, quando, filtro).
 - Retenção: pessoa removida mantém conta e histórico próprios; o vínculo com a empresa fica marcado como encerrado, não apagado.
-
-

10. Backlog recomendado (depois do núcleo, em ordem de valor)

1. **Trilhas por área** (playbooks de Marketing, Financeiro, RH, Jurídico, Comercial, Operações) com gate por faixa. O catálogo atual é 100% dev-centric — é a maior lacuna para funcionário não-técnico.
2. **Turmas/cohorts** — Fundação mensal com calendário, presença e gravação. Resolve entrada em datas diferentes sem depender de agenda de fundador.
3. **Certificado e selo compartilhável** por faixa (imagem + link verificável + post pronto de LinkedIn). Marketing orgânico gratuito, feito pelo aluno.
4. **Agente de WhatsApp** — lembrete, dúvida, submissão de build por áudio/foto, resumo semanal. Reaproveitar a stack de WhatsApp que já roda na infra da Águia.
5. **Diagnóstico por área** (Builder Score Zero) — questionário por departamento antes do início: responsável, tamanho do time, horas perdidas, sistemas, dor 0–12, três quick wins. Destrava venda e vira a linha de base.
6. **Biblioteca de builds reutilizáveis** — build de uma empresa disponível (com permissão) como template para outra. É o efeito de rede do produto.
7. **Pré-triagem de artefatos por IA** — a IA aplica a rubrica e escreve o rascunho do parecer; humano decide. É o que segura o SLA de 5 dias quando escalar.

- 8. **SSO/SAML + verificação de domínio** — exigência de empresa grande.
 - 9. **Cobrança por assento** (quantity no gateway) — hoje o contrato é faturado fora e provisionado na mão.
 - 10. **API de leitura + webhooks do Score** — cliente puxa para o BI dele; e o Success monta relatório sem scraping.
 - 11. **Painel do líder de área** — recorte do painel para o gestor direto (o gestor é quem cobra de verdade).
 - 12. **Detecção de assento zumbi** — assento sem uso há 30 dias vira sugestão de realocação para outra pessoa da empresa. Aumenta valor percebido do contrato sem vender mais nada.
-

11. Ordem de implementação

#	MÓDULO	ESTIMATIVA	POR QUE NESSA ORDEM
0	Descoberta: mapa do schema atual, decisão sobre teams do Stack Auth, helper único de entitlement	2–3 dias	Bloqueia tudo; risco é de acoplamento invisível
1	M1 — empresa, assentos, áreas, convites, wizard de onboarding	6–9 dias	Base de todo o resto e a dor operacional de hoje
2	M2 — perfil/identidade + onboarding com roteamento	3–4 dias	Sem isso não existe clusterização
3	M3a — ledger de eventos + pontos + streak + perfil	5–7 dias	Instrumentação: todo módulo seguinte lê daqui
4	M4 — painel da empresa (org_admin e sponsor)	5–7 dias	Primeiro entregável que o cliente vê
5	M3b — faixas: artefatos, fila de revisão, awards	8–12 dias	Maior fase; é workflow humano com SLA, não barra de progresso
6	M5 — admin CB: filtros e audiências	4–6 dias	Depende de M2 + M3a
7	M6 — feed com escopo	4–6 dias	Alto valor percebido, baixo acoplamento; pode andar em paralelo
8	Builder Score + snapshots	3–5 dias	Precisa de M3b para contar artefato

Total aproximado: **35–55 dias de desenvolvimento** após a descoberta, com um desenvolvedor. Cada módulo entra atrás de feature flag e pode ir para produção sozinho.

Riscos: (1) qualquer atalho do tipo “faço fora do hub com login próprio” vira dívida permanente — não fazer; (2) fila de revisão sem dono definido vira gargalo (o SLA é de gente, não de código); (3) acoplamento de billing/entitlement ao plano individual pode ser maior do que parece de fora — por isso a descoberta vem antes de qualquer estimativa virar compromisso.

12. Perguntas que precisamos responder na primeira call

1. **Schema atual:** dump de usuário, assinatura, progresso de aula, projeto/challenger, badges, onboarding. Onde já existe algo parecido com empresa/organização?
 2. **Stack Auth:** a versão usada tem *teams/organizations* nativos? Se tiver, M1 encurta bastante — mas o vínculo de negócio (assento, área, cargo) continua no nosso banco.
 3. **Entitlement:** hoje o acesso é concedido por rota a partir do plano individual (`features []. route_path`). Qual o menor caminho para adicionar concessão por empresa sem reescrever o checkout?
 4. **Billing:** dá para representar quantidade (assentos) no gateway atual, ou seguimos com fatura fora + provisionamento manual no começo?
 5. **Feed:** existe alguma estrutura de post/comunidade hoje, ou M6 nasce do zero?
 6. **Notificações:** qual o canal já implementado (e-mail transacional, push, WhatsApp)? Reaproveitar, não montar outro.
 7. **GitHub OAuth:** alguma restrição para adicionar um provider de conexão (não de login) ao perfil?
 8. **Jobs agendados:** existe infraestrutura de cron/worker no deploy atual para os snapshots semanais e o resumo de segunda-feira?
 9. **Ambientes:** existe staging? Precisamos de um para testar importação de CSV com dado real de cliente sem risco.
 10. **Os dois SKUs B2B quebrados no checkout público** (R\$ 297 sem método de pagamento e R\$ 499,90 expirado): consertar ou remover?
-

Cultura Builder · especificação interna · 20/08/2026. Contexto de produto e metodologia: [metodo.brunopessoa.com](#) (deck), [/plano.html](#) (plano B2B completo), [/briefing-produto.html](#) (spec F0–F4 anterior, que este documento substitui e amplia).

